



MTA

AUDIO EDITOR

User Manual

Complete guide to using MTA Audio Editor

Version: 0.2.0 · Revision: 159

Build: 20261006021358

Alessandro De Salvo

braket71@gmail.com

<https://github.com/desalvo/mta-audio-editor>

License: EUPL-1.2

Digital Audio Workstation with MTA support



Windows



macOS



Linux



Android



iOS

VST3 Support Manual

VST3 support for MTA Audio Editor

MTA Audio Editor 0.3.0-r12 — Early release

Overview

MTA Audio Editor 0.3.0 offers experimental offline VST3 track inserts on desktop. The current pipeline uses optional Pedalboard processing and an isolated C++ probe for factory inspection. This is NOT yet a fully featured realtime VST3 host.

Platforms

Windows, macOS, and Linux desktop. Each plugin must match the application CPU architecture (x86_64 or ARM64). Desktop VST3 hosting is not available in the web or mobile applications.

Installation and licensing

Install each VST3 separately in accordance with its vendor license. MTA does not distribute commercial plugins or embed them in project files. Install the optional offline backend in the Python environment actually used by MTA: `pip install -r requirements-vst3.txt`. Native bundles may not include it by default.

Directories and scanning

Plugins are discovered in the operating system's standard VST3 directories. Add custom locations using `MTA_VST3_PATHS` and the platform path separator. Reopen the Add VST3 dialog to rescan. Check CPU architecture and external plugin dependencies when a plugin is missing.

Using mixer inserts

Open a project, select a channel, click Insert, and choose Add VST3 (desktop). Select a discovered plugin and edit the parameters exposed by the plugin (normalized to 0-1). The project stores the plugin location and settings, not its binary. The VST3 is applied when rendering the individual track.

Effect ordering

The offline pipeline runs built-in inserts before enabled VST3 inserts. If a built-in insert is positioned after an enabled VST3, MTA raises an explicit error rather than silently changing the chain order. Use bypass to compare processed and dry audio.

Native probe and safety

The C++20 factory probe checks `GetPluginFactory` and, when built against the Steinberg SDK, can inspect class names and categories. It runs out of process with a timeout. Every VST3 executes third-party native code; install only trusted plugins. The probe does NOT provide live audio hosting.

Limitations in 0.3.0-r12

Not yet supported: original VST3 editor windows, SDK-based low-latency realtime processing, MIDI events, VST3 automation, plugin delay compensation, and Master-channel VST3 processing. Use built-in inserts for unsupported scenarios. Discovery alone is not evidence of compatibility.

Troubleshooting

Plugin missing: verify scan paths and architecture. Render errors: verify plugin dependencies and optional Pedalboard installation. GUI will not open: original plugin editors are not supported yet. When moving projects to another machine, install the plugin on the destination; the project does not contain it.

Licenses and redistribution

- MTA Audio Editor: EUPL-1.2.
- Steinberg VST3 SDK (MIT-licensed releases): MIT.
- Pedalboard (optional VST3 backend): GPL-3.0.

r10: isolated instance creation

With the independently installed VST3 SDK, run ``mta_vst3_probe /path/to/module --instantiate <32-hex-CID>`` in the diagnostic subprocess. The component is released immediately; this does not initialize the plugin, process audio or enable realtime hosting.

r11: isolated lifecycle diagnostic

``mta_vst3_probe /path/to/module --lifecycle <32-hex-CID>`` tries initialization with a null host context and termination only after successful initialization. Some plugins will reject this context. This does not activate audio, GUI, MIDI, or realtime playback.

r12: VST3 bus diagnostic

After successful initialization, the isolated probe enumerates audio and event buses, direction, and channel counts without processing audio or activating the plugin.